LECTURE 8 · CS-401 PARALLEL AND DISTRIBUTED COMPUTING

Memory Hierarchy and Memory Consistency

How modern processors store, access, share and update data — and why it matters for the performance and correctness of parallel programs.



LEARNING OUTCOMES

By the end of this lecture, you can…

01 Explain the memory hierarchy: registers, cache, RAM and secondary storage.

02 Differentiate cache hits from cache misses.

03 Explain temporal and spatial locality.

04 Describe cache coherence in multicore processors.

05 Understand the basic MESI cache-coherence protocol.

06 Differentiate cache coherence from memory consistency.

07 Explain sequential consistency and memory ordering with simple examples.

08 Identify how memory behavior affects parallel-program performance and correctness.

INTRODUCTION

Introduction to the memory hierarchy

- A computer needs memory to store **instructions and data**.
- Memory types differ in **speed, capacity and cost**.
- Fast memory is generally smaller and more expensive per byte.
- Large-capacity memory is generally slower.
- The memory hierarchy **balances speed, capacity and cost**.

Example: a processor repeatedly adding numbers keeps the values it needs in registers or cache instead of fetching them from RAM every time.

ANALOGY

Your desk and your bookshelf

Books you use often stay on the desk: fast to reach, but space is limited. The rest wait on the shelf: more room, slower to fetch. Computers work the same way: small, fast memories hold frequently used data, while larger, slower memories store everything else.

MOTIVATION

Why do we need a memory hierarchy?

ONE MEMORY TECHNOLOGY CANNOT EASILY PROVIDE ALL THREE

Extremely high speed

Data delivered as fast as the CPU can use it.

Very large capacity

Room for programs, datasets and files.

Very low cost

Affordable per byte at a large scale.

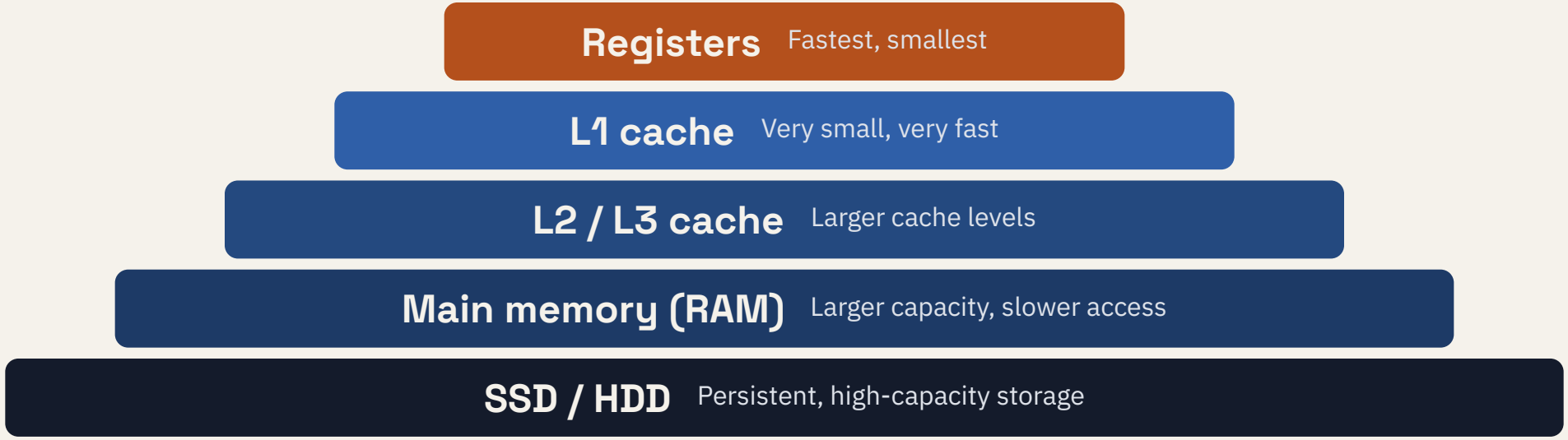
Everything in very fast memory → the system would be **far too expensive**.

Everything in cheap, slow storage → the CPU would spend its time **waiting for data**.

The hierarchy combines technologies and keeps frequently used data **close to the processor**, reducing the **average** time needed to access data.

THE BIG PICTURE

The memory hierarchy



↑ Faster · smaller · more costly per byte

↓ Larger · slower · cheaper per byte

Conceptual hierarchy — exact arrangements vary by system.

LEVEL 1

Registers

- Small storage locations **inside the CPU**.
- Hold values needed **immediately** by instructions.
- Examples: general-purpose registers and the program counter.
- Very fast access, but very limited capacity.

EXAMPLE: COMPUTING 10 + 20

```
int a = 10;
int b = 20;
int sum = a + b;
```

The values may be loaded into registers, added, and the result stored in a register — the CPU operates directly on register values.

The compiler and processor decide which variables live in registers; it is **not guaranteed** for every variable.

LEVEL 2

Cache memory

- A small, fast memory **close to the processor**.
- Stores **copies** of data from main memory.
- Reduces the need to access RAM repeatedly.
- Commonly organized into **L1, L2 and L3** levels.

Think of it as a small workspace

It holds information the CPU is likely to need again. If the data is already there, the CPU reaches it much faster than going to RAM.

```
for (int k = 0; k < 100; k++)
    for (int i = 0; i < 8; i++)
        total += A[i]; // reused 100x
```

After the first pass, A[0]...A[7] are likely in cache, so later passes are cheap.

CACHE LEVELS

L1, L2 and L3 cache

PRIVATE · PER CORE

L1

Usually the smallest and fastest cache, close to a CPU core. Often split into separate instruction and data caches.

OFTEN PRIVATE

L2

Typically larger than L1, with somewhat higher access latency. Often private to one core.

OFTEN SHARED

L3

Often larger and shared among multiple cores, depending on the processor design.

A TYPICAL LOOKUP PATH WHEN DATA IS NOT FOUND



Exact arrangements and behavior differ across processors.

LEVEL 3

Main memory (RAM)

- Stores the **programs and data currently in use**.
- Greater capacity than CPU caches.
- Generally slower to access than cache.
- **Volatile:** contents are normally lost when power is removed.

IN PRACTICE

Example: opening a large dataset

The operating system loads the program and the relevant data into RAM. The CPU then accesses portions of that data through the memory system — registers and cache.

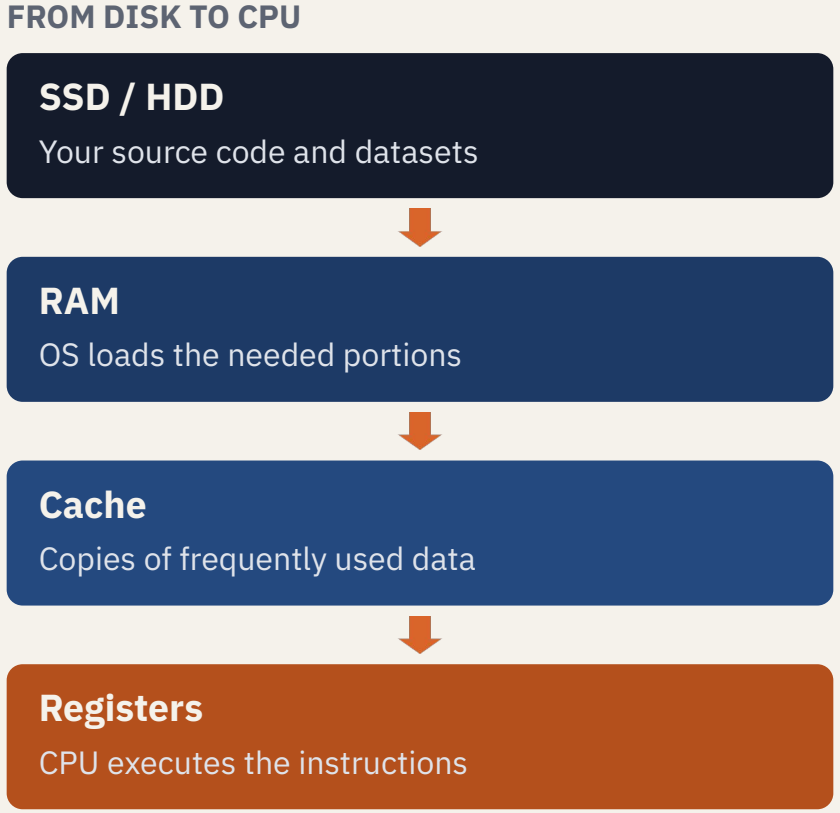
If the CPU repeatedly needs data that is **not in cache**, it must wait while the memory subsystem retrieves it from RAM.

LEVEL 4

Secondary storage

- **Examples:** Solid-State Drive (SSD) and Hard Disk Drive (HDD).
- Stores data **persistently**.
- Much greater capacity than registers and caches.
- Generally slower to access than RAM.
- Holds the operating system, applications and files.

> ⚠ An SSD is **not** just another cache level: it is persistent storage managed through the operating system.



SUMMARY

Comparing memory types

MEMORY TYPE	RELATIVE SPEED	TYPICAL CAPACITY	PERSISTENT?
Registers	Extremely fast	Very small	No
Cache	Very fast	Small	No
RAM	Faster than secondary storage	Medium to large	No
SSD / HDD	Slower access	Large	Yes



These are general relationships, not exact timings. The CPU performs best when the data it needs is already in **registers or cache**, rather than in slower levels.

CACHE PERFORMANCE

Cache hits and cache misses

Cache hit

The requested data **is found** in the cache being checked.

Cache miss

The requested data **is not found** there, so it must come from another memory level.

01
CPU requests a value

→

02
Cache checks for the data

→

03
Found → a hit

→

04
Not found → fetch from next level

Desk analogy: the book is on your desk (hit) or you walk to the shelf (miss). An **L1 miss does not necessarily mean RAM**: the data may be in L2 or L3.

CACHE PERFORMANCE

Hit ratio and miss ratio

Hit Ratio = Cache Hits ÷ Total Memory Accesses

Miss Ratio = 1 − Hit Ratio

WORKED EXAMPLE

A program performs **1,000** memory accesses. **900** of them are hits at the cache level being measured.



Hit ratio = 900 ÷ 1000 = 0.90 = **90%**

Miss ratio = 1 − 0.90 = **10%**

CACHE PERFORMANCE

Average memory access time (AMAT)

AMAT = Hit time + (Miss ratio × Miss penalty)

`Hit time`	time to access the cache on a hit
`Miss ratio`	fraction of accesses that miss
`Miss penalty`	additional time spent on a miss

A simplified single-cache model; real multilevel caches need more detailed calculations.

WORKED EXAMPLE

Hit time = **2 ns**

Miss ratio = 10% = **0.10**

Miss penalty = **50 ns**

```
AMAT = 2 + (0.10 × 50)
AMAT = 2 + 5
```

AMAT = 7 ns

LOCALITY

Temporal locality

If a program accesses a memory location, it is likely to access the same location again soon.

```
int total = 0;

for (int i = 0; i < 1000; i++)
{
    total = total + i;
}
```

- **total** is read and updated **1,000 times**.
- Keeping it close to the processor, in a register or cache, makes every reuse cheap.
- Rule of thumb: recently used data is likely to be used again.

LOCALITY

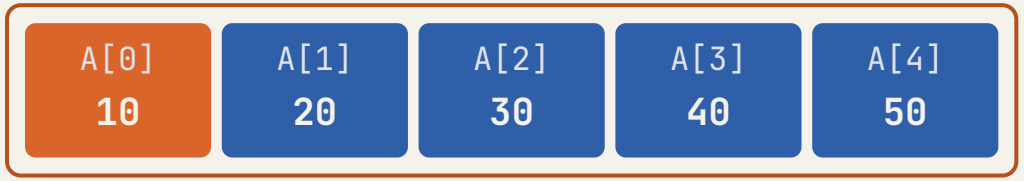
Spatial locality

When a program accesses one memory location, it is likely to access nearby locations soon.

```
int A[5] = {10, 20, 30, 40, 50};

for (int i = 0; i < 5; i++)
{
    printf("%d\n", A[i]);
}
```

ONE CACHE LINE (CONCEPTUAL)



Data moves between memory and cache in blocks called **cache lines**. Fetching A[0] may also bring A[1]…A[4] into cache, so the next accesses are hits.

PARALLEL PERFORMANCE

Why the memory hierarchy matters in parallel computing

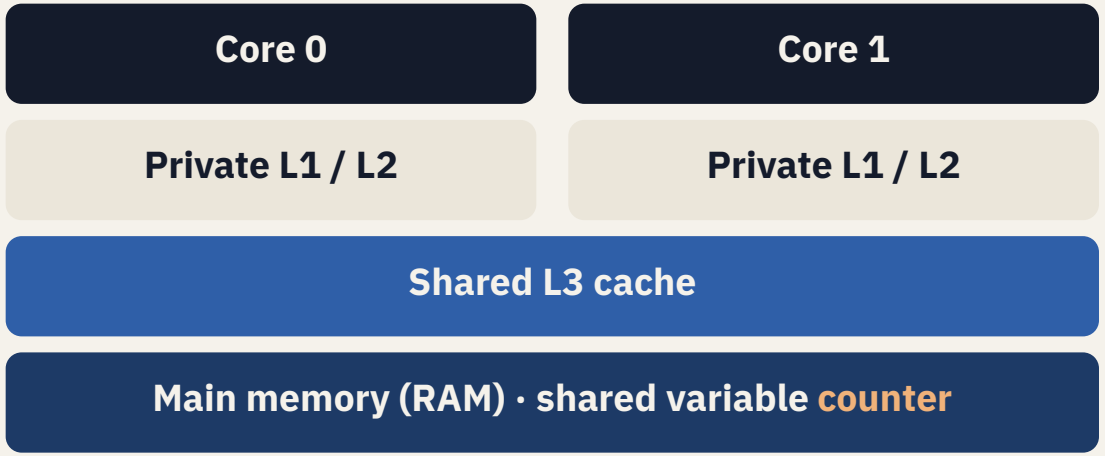
- Multiple threads or processes may repeatedly access data.
- Cache misses increase execution time.
- Data layout and access patterns affect performance.
- Shared caches can create contention between cores.
- Parallel speedup depends on computation **and** memory behavior.

More threads ≠ automatically faster

If threads spend most of their time waiting for data from memory, the speedup is limited. Efficient parallel programs consider both how work is divided and how data is accessed.

SHARED MEMORY

Multicore processors and shared memory



- Each core executes instructions **independently**.
- Cores may have **private caches**.
- Cores may share some cache levels and main memory.
- Multiple cores may access the **same shared variables**.

⚠ If Core 0 changes **counter**, Core 1 must not keep using an outdated cached copy. This is the problem of **cache coherence**.

CACHE COHERENCE

What is cache coherence?

The mechanism by which a multicore system maintains a coherent view of copies of the same memory location.

APPROACH 1

Snooping-based

Caches observe coherence traffic on a shared interconnect and react to other cores' reads and writes.

APPROACH 2

Directory-based

A directory records which caches hold each memory block and messages only those caches.

PROTOCOL

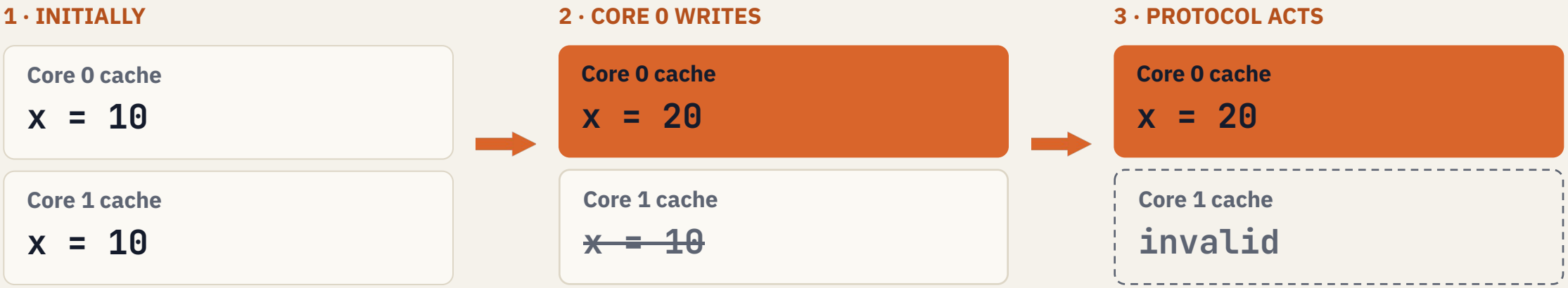
Protocols such as MESI

Define the states of a cache line and the transitions on reads and writes.

Coherence stops cores from using stale values indefinitely. It is managed by **hardware**, but programmers must still handle **synchronization and data races**.

CACHE COHERENCE

A cache-coherence example



The hardware may **invalidate** or **update** the other copy, depending on the design.

Coherence doesn’t mean every cache always holds the same value: reads and writes follow the **protocol’s rules**.

TWO DIFFERENT IDEAS

Cache coherence vs memory consistency

	CACHE COHERENCE	MEMORY CONSISTENCY
Scope	Accesses to an **individual memory location**.	Ordering and visibility of operations **across locations**.
Job	Coordinates copies of the same data across caches.	Defines which observations of concurrent reads and writes are allowed.
Defined by	Often implemented by hardware protocols.	The processor architecture and programming model.

Coherence asks: if two cores cache the same variable, how are their copies kept coherent?

Consistency asks: if one core writes two different variables, in what order may another core see them?

Introduction to the MESI protocol

MESI is a cache-coherence protocol named after the **four states** a cache line can be in. It helps coordinate cached copies when processors read or write shared memory.

M
Modified
Changed in this cache; memory is out of date.

E
Exclusive
The only cached copy; matches memory.

S
Shared
May be in several caches; clean.

I
Invalid
Not valid; must be fetched again.

The next slides explain each state with a simple example.

MESI STATE

Modified (M)



- The cache line has been **modified** by this cache.
- Its contents **differ** from the value in the next memory level.
- This cache holds the **only writable copy** (basic MESI model).
- The data must eventually be **written back** or propagated.

EXAMPLE

A cache line holds x = 10. Core 0 writes x = 20, so the line enters **M**. The next memory level may still hold 10 until the change is propagated.

MESI STATE

Exclusive (E)



- The cache line is present in **only one cache**.
- It **matches** the value in memory (it is clean).
- A local write can move it to **Modified** without invalidating any other copy.

EXAMPLE

Core 0 reads x = 10 and no other cache holds the line, so it is held in **E**. If Core 0 then writes, the line can move **E → M** without contacting other caches.

MESI STATE

Shared (S)



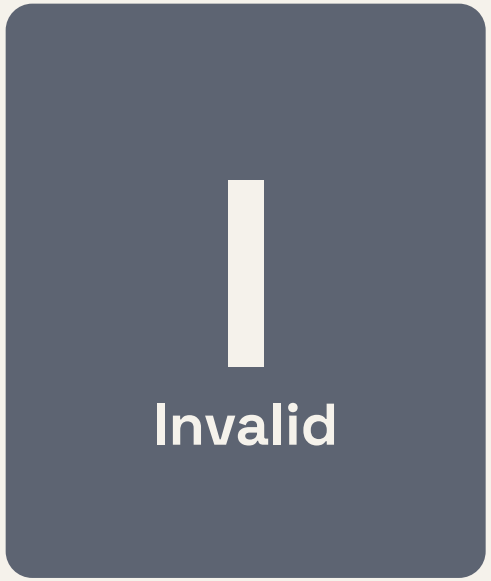
- The cache line **may exist in several caches** at once.
- The data is **clean**: consistent with the next memory level.
- To write, a core must first gain **exclusive ownership**, invalidating the other copies.

EXAMPLE

Core 0 and Core 1 both read x = 10, so both caches hold the line in **S**. If Core 0 wants to change x, it must first get permission and coordinate with Core 1’s cache.

MESI STATE

Invalid (I)



- The cache line is **not currently valid** for use.
- It may **never have been loaded** into this cache, or...
- ...it was **invalidated** because another core needed to modify it.
- A later access must **fetch a valid copy** again.

EXAMPLE

Core 0 modifies a shared line, so Core 1's copy becomes **I**. Core 1 cannot treat it as current data; its next read obtains a valid value through the coherence system.

MESI IN ACTION

MESI example: two cores

STEP	EVENT	CORE 0	CORE 1	VALUE
1	Core 0 reads x (no other cache has it)	E	I	x = 10
2	Core 1 reads x — both now share the line	S	S	x = 10
3	Core 0 writes x = 20: gets write permission, Core 1’s copy is invalidated	M	I	x = 20
4	Core 1 reads x again: obtains the valid value through the coherence system	S	S	x = 20

Exact messages and transitions depend on the implementation. The principle: a core **never keeps using an invalidated copy** as if nothing changed.

MEMORY CONSISTENCY

What is memory consistency?

Memory consistency defines the ordering and visibility rules for memory operations performed by multiple processors or threads.

Can one thread observe a write **before** another write?

When does a write become visible to another thread?

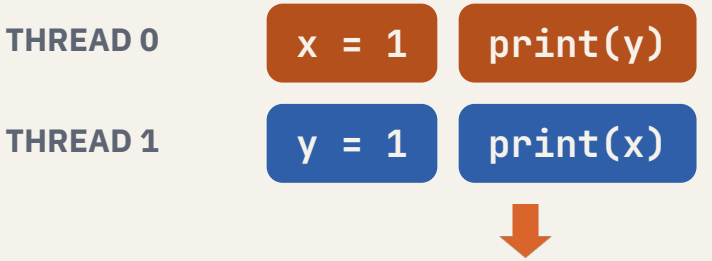
Which **ordering guarantees** does the processor or programming model provide?

Even if each variable is kept coherent, different cores may observe operations on **different variables** in different orders, unless ordering guarantees exist.

Sequential consistency

- All threads' memory operations appear in **one global order**.
- That order respects each thread's own **program order**.
- Each read observes the **latest write** before it in that global order.

A **conceptual model**: real hardware may give weaker guarantees unless synchronization or other ordering mechanisms are used.



ONE POSSIBLE GLOBAL ORDER



Operations from different threads may **interleave**, but each thread's own order is kept.

Example: can both threads print 0?

Initially **x = 0** and **y = 0**. Two threads run at the same time:

THREAD 0

```
x = 1;
print(y);
```

THREAD 1

```
y = 1;
print(x);
```

Under sequential consistency: No.

In any global order, the **last** of the four operations is a print, and it comes after **both** writes. So at least one thread must print 1.

Why “both print 0” forms a cycle

1	T0 writes x = 1	before	T0 reads y	Program order of Thread 0
2	T0 reads y → gets 0	before	T1 writes y = 1	To read 0, it must come before the write
3	T1 writes y = 1	before	T1 reads x	Program order of Thread 1
4	T1 reads x → gets 0	before	T0 writes x = 1	To read 0, it must come before the write

⚠ Step 4 leads back to step 1: a **circular ordering**. No single global order can satisfy it, so (0, 0) is impossible under sequential consistency.

Memory reordering

Processors and compilers may reorder certain memory operations, when the rules allow it, to improve performance.

- You write operations in a particular order.
- The compiler or processor may execute or expose them **differently**.
- What is permitted depends on the **memory model**.
- **Synchronization** can impose the ordering you need.

How does this happen?

Store buffers let a core continue before its write is visible to others. **Out-of-order execution** runs independent instructions early.
With a store buffer, both threads in our x/y example **can print 0** on real hardware such as x86.

Memory ordering and synchronization

Mutexes & locks
Only one thread at a time enters a critical section.

Atomic operations
Indivisible read-modify-write, e.g. counter++.

Semaphores
Count-based access control and signalling.

Barriers
Every thread waits until all have arrived.

Memory fences
Force an order on memory operations where needed.

⚠ These prevent data races and establish ordering, e.g. when one thread writes data and then signals another. A **delay (sleep) or an ordinary print is not** reliable synchronization.

PRACTICE

Shared counter: a race condition

```
counter = counter + 1;
```

LOOKS LIKE ONE STEP, BUT IT IS THREE

1 · Read **2 · Add one** **3 · Write**

⚠ Cache coherence does **not** make the read-modify-write sequence atomic. Use a **mutex** or an **atomic increment**.

TIME	THREAD A	THREAD B	COUNTER
t1	read → 0		0
t2		read → 0	0
t3	add → 1		0
t4		add → 1	0
t5	write 1		1
t6		write 1	1

Final value: 1 — but two increments were intended (expected **2**).

PRACTICE

Correcting the counter with OpenMP

```c
#include <stdio.h>
#include <omp.h>

int main(void)
{
    int counter = 0;

    #pragma omp parallel for
    for (int i = 0; i < 10000; i++)
    {
        #pragma omp atomic update
        counter++;
    }

    printf("Final counter = %d\n", counter);
    return 0;
}
```

COMPILE AND RUN (GCC)

```
gcc -fopenmp counter.c -o counter
./counter
```

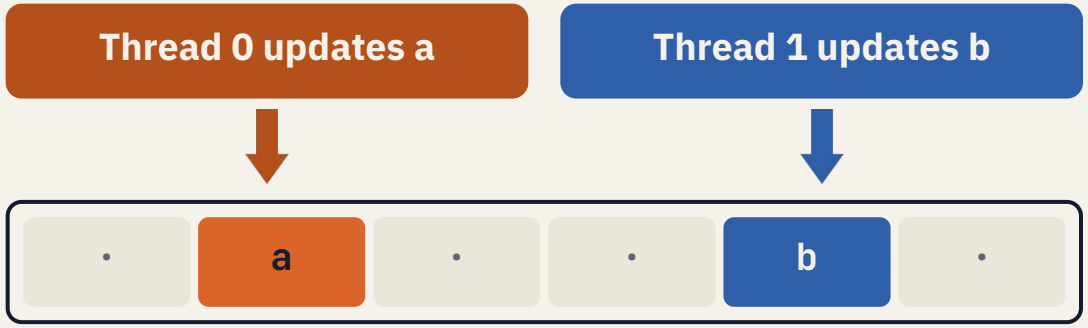
EXPECTED OUTPUT

```
Final counter = 10000
```

Iterations run in parallel, but each increment is **atomic**, so no thread loses another thread's update.

PERFORMANCE PITFALL

False sharing



One cache line. Every write by one core invalidates the other core's copy, so the line "ping-pongs" between caches.

- Different threads update **different variables**...
- ...that happen to sit in the **same cache line**.
- The coherence protocol repeatedly transfers or invalidates the line.
- Performance suffers, although no variable is logically shared.

FIX IT

Solutions

- Change the data layout.
- Separate frequently updated variables (e.g. padding).
- Use per-thread data and combine at the end.

OPTIMIZATION

Improving memory performance

1 **Access arrays sequentially**
Walk memory in order so each cache line is fully used.

2 **Reuse data**
Do more work on data while it is still in cache.

3 **Organize data for spatial locality**
Store data that is used together, together.

4 **Reduce unnecessary transfers**
Avoid copying or re-reading data you already have.

5 **Avoid false sharing**
Keep different threads' hot variables on different lines.

6 **Synchronize only where required**
Locks and atomics cost time; protect shared updates only.

7 **Measure before and after**
The best optimization depends on workload and hardware.

Good parallel programming is not just using more cores — it is **using memory efficiently**.

CONNECTING THE COURSE

Connection to MPI and OpenMP

SHARED MEMORY

OpenMP · threads

Threads share a common address space and may share caches, so data races, coherence and false sharing matter.

DISTRIBUTED MEMORY

MPI · processes

Processes communicate through messages and usually have separate address spaces. One process cannot directly read another’s private variables.

BOTH

Hybrid

MPI coordinates processes across nodes; OpenMP manages threads within a node. You must handle both communication and synchronization.



Cache coherence is especially relevant to **shared-memory hardware**; memory ordering and synchronization matter in **every** parallel programming model.

CHECK YOUR UNDERSTANDING

Review questions

1. Why can’t a computer use only the fastest type of memory?
2. A cache has a 2 ns hit time, a 5% miss ratio and a 60 ns miss penalty. What is the AMAT?
3. Give one example of temporal locality and one of spatial locality.
4. Two cores hold x in the Shared state. What happens when Core 0 writes x?
5. What is the difference between cache coherence and memory consistency?
6. Why can two threads incrementing a shared counter give the wrong result, and how do you fix it?

RECAP

Key takeaways

1. The hierarchy trades **speed, capacity and cost**; keeping data close to the CPU lowers the average access time.
2. Hits, misses and **locality** drive cache performance: AMAT = hit time + miss ratio × miss penalty.
3. **Coherence** keeps copies of one location coherent (e.g. MESI); **consistency** defines ordering across locations.
4. Coherence is not synchronization: use **atomics, locks and barriers**, and watch for false sharing.

Questions?